Git

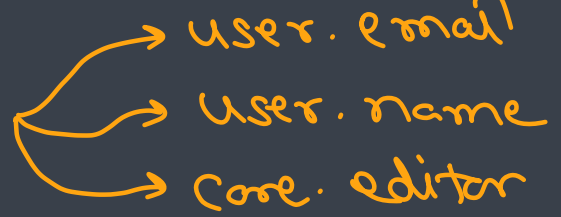
Basic Workflow

Ubuntu install → sudo apt install git

Windows install → download git bash & install.

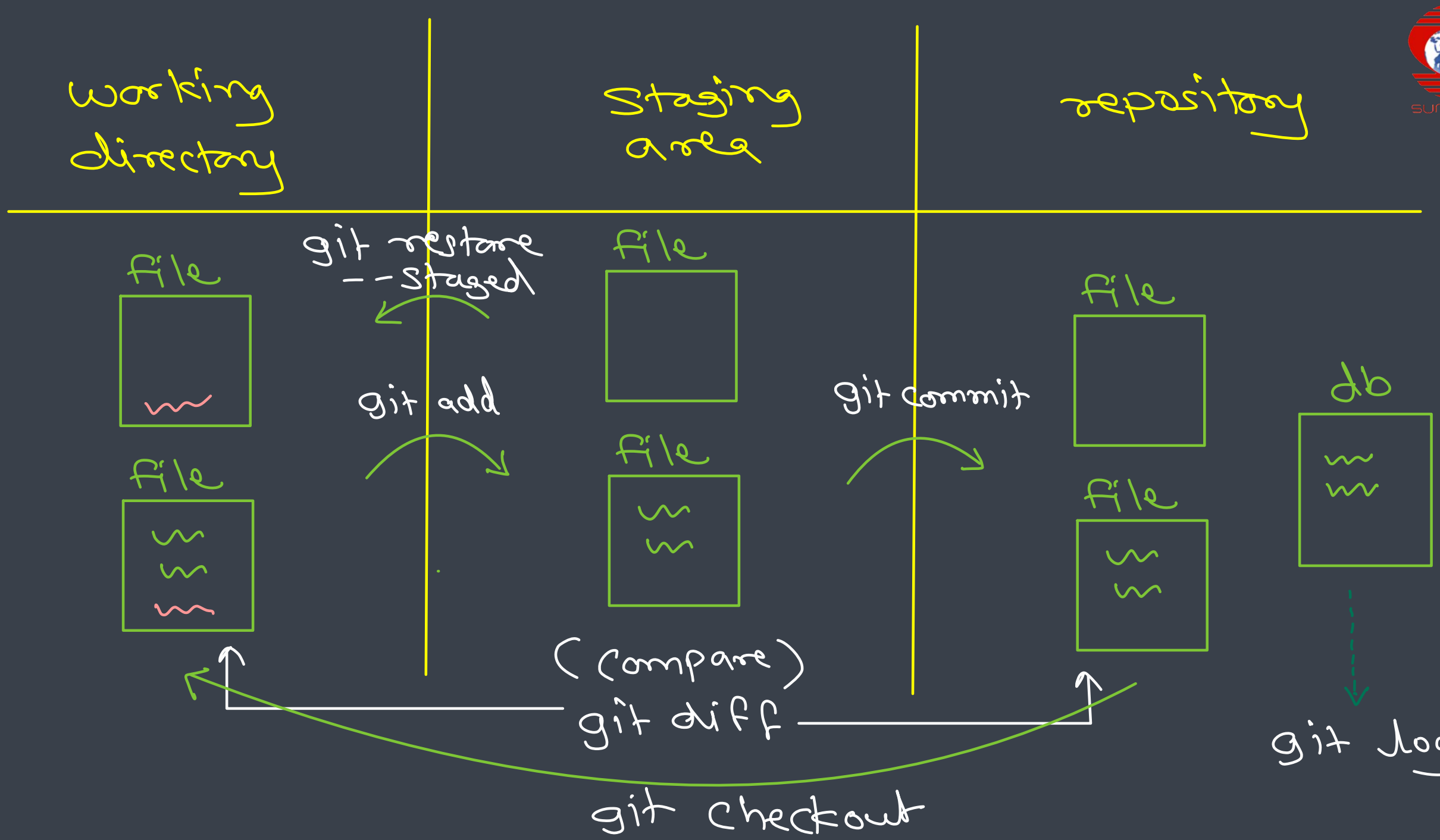
git config

- The git config command is a function that sets configuration variables
- It controls git look and operation
- Levels
  - Local (--local)
    - When no configuration option is passed git config writes to a local level, by default
    - The repository of the .git directory has a file that stores local configuration values
  - Global (--global) → ~/.gitconfig
    - The application of the global level configuration includes the operating system user
    - Global configuration values can be found in a file placed in a user's home directory
  - System (--system)
    - The System-level configuration includes all users on an operating system and all repositories
    - System-level configuration file is located in a git config file of the system root path



git init

- The git init command is used to generate a new, empty Git repository or to reinitialize an existing one
- With the help of this command, a .git subdirectory is created, which includes the metadata, like subdirectories for objects and template files, needed for generating a new Git repository


working directory
Staging area
repository
git restore --staged
file
file
file
file
file
file
git add
git commit
db
(compare)
git diff
git checkout
git log

git add

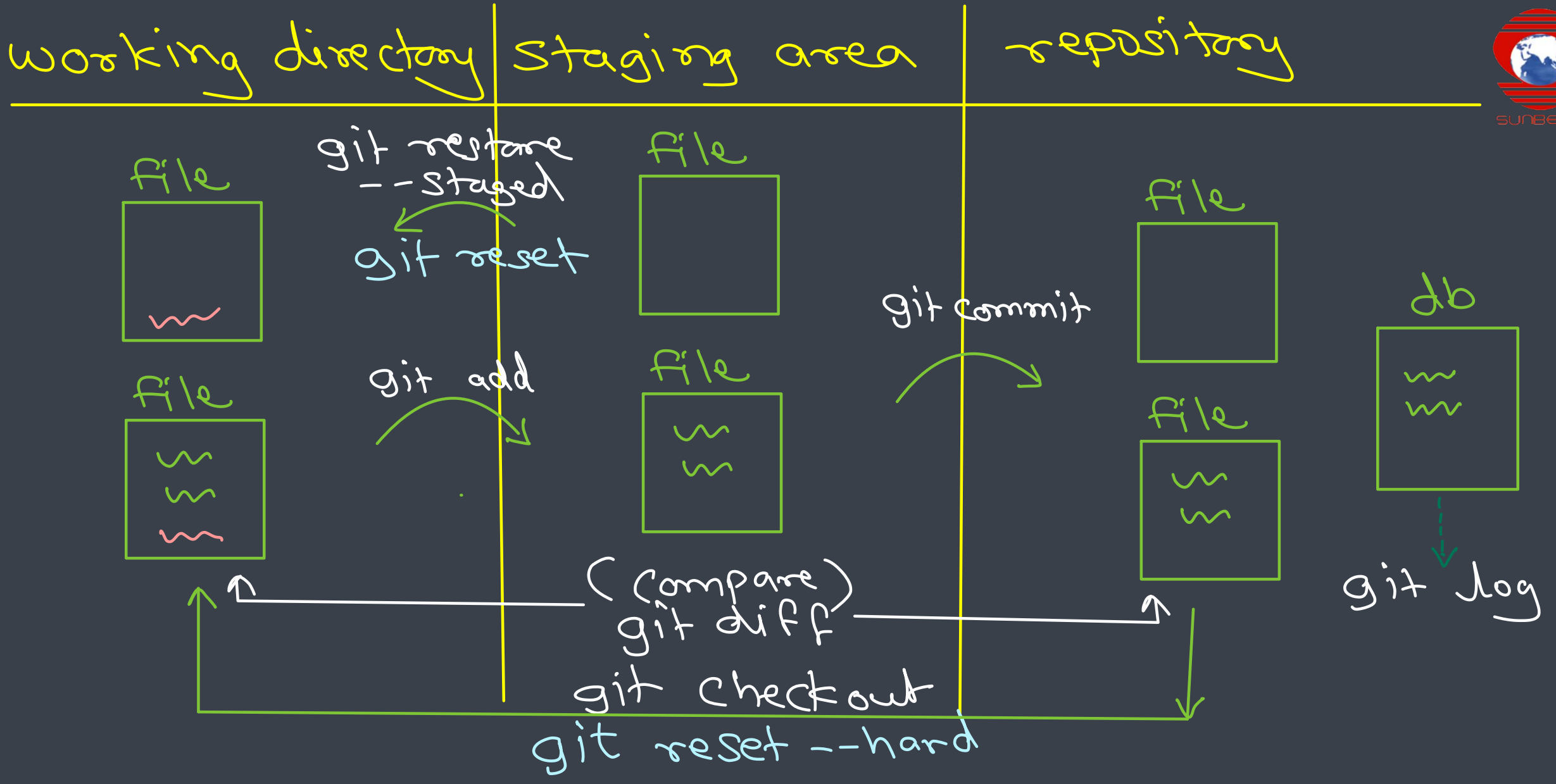
- The git add is a command, which adds changes in the working directory to the staging area
- With the help of this command, you tell Git that you want to add updates to a certain file in the next commit
- But in order to record changes, you need to run git commit too
- In combination with the commands mentioned above, git status command is also needed to see which state the working directory and the staging area are in

git commit

- The git commit command saves all currently staged changes of the project
- Commits are created to capture the current state of a project
- Committed snapshots are considered safe versions of a project because Git asks before changing them
- Before running git commit command, git add command is used to promote changes to the project that will be then stored in a commit
- Working of commit
  - Git snapshots are committed to the local repository
  - Git creates an opportunity to gather the commits in the local repository, rather than making a change and commit it immediately to the central repository
  - This has many advantages splitting up a feature into commits, grouping the related commits, and cleaning up local history before committing it to the central repository
  - This also gives the developers an opportunity to work in an isolated manner

git log

- The git log command shows committed snapshots
- It is used for listing and filtering the project history, and searching for particular changes
- The git log only works on the committed history in comparison with git status controlling the working directory and the staging area


working directory
staging area
repository
file
file
file
file
file
db
git restore --staged
git reset
git add
git commit
(compare)
git diff
git checkout
git reset --hard
git log

Branching

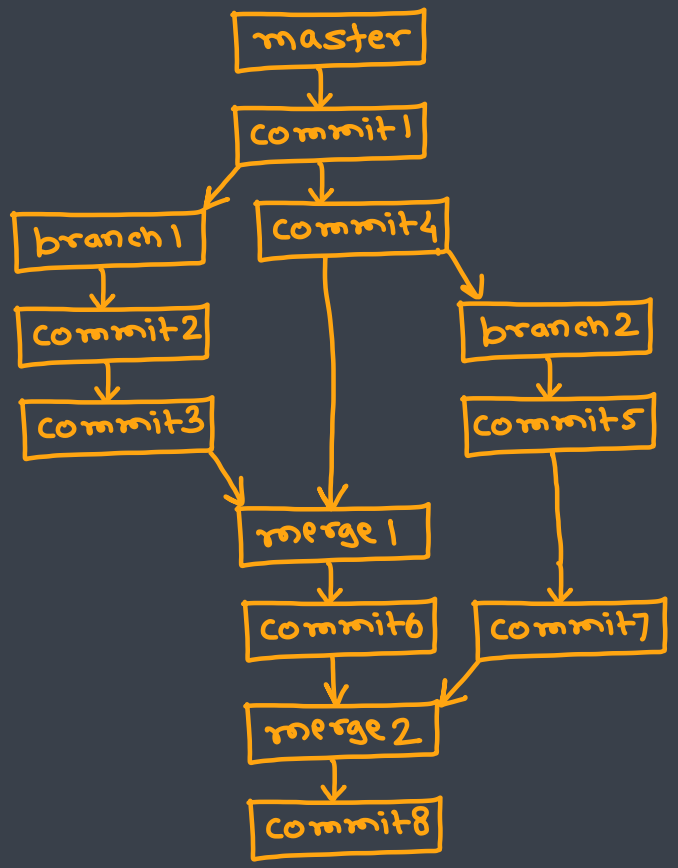
Branching

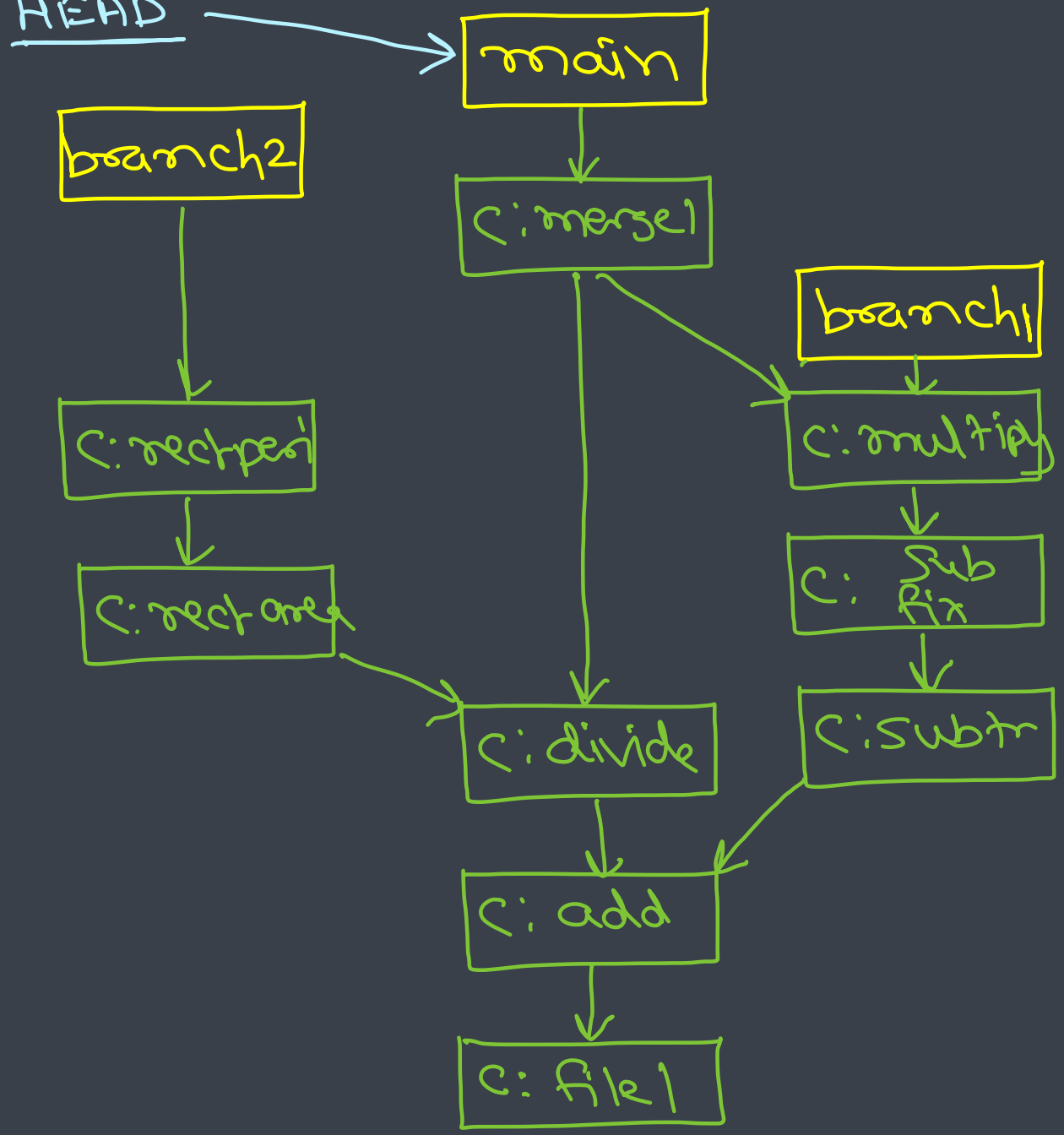
- Branching allows developers to branch out from the original code base and work separately
- Allows another line of development
- A way to write code without affecting the rest of your team
- Generally used for feature development
- Once confirmed the feature is working you can merge the branch in the master branch and release the build to customers
- Why is it required ?
  - So that you can work independently
  - There will not be any conflicts with main code
  - You can keep unstable code separated from stable code
  - You can manage different features keeping away the main line code and there wont be any impact of the features on the main code

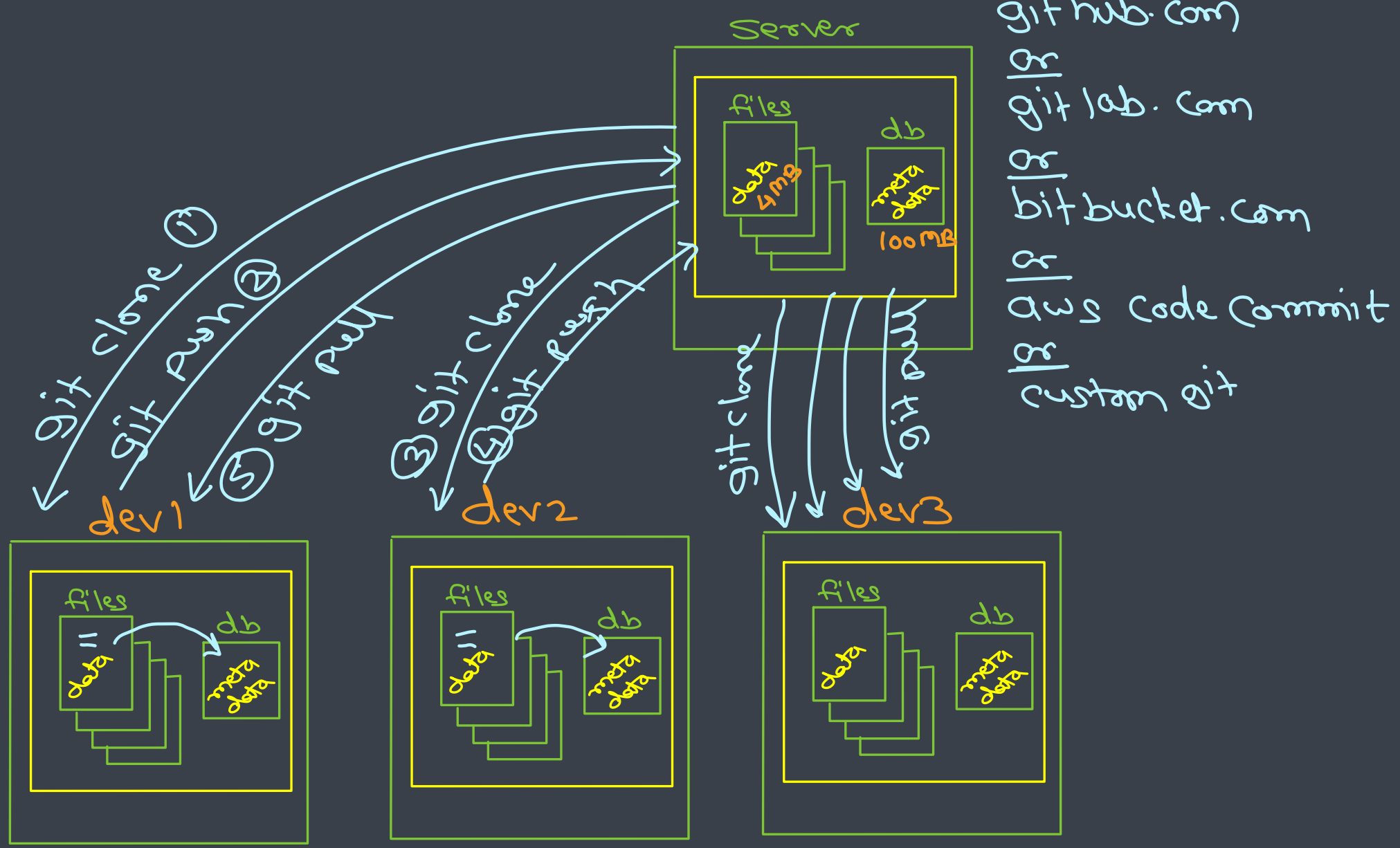
```
> git branch          ──→ list all branches
> git branch b1       ──→ create "b1" branch
> git checkout b1     ──→ switch to "b1" branch
> git checkout -b b2  ──→ create branch b2 & switch to it
> git checkout main   ──→ switch to main branch
> git branch -d b1    ──→ delete 'b1' branch
> git merge b2        ──→ merge 'b2' branch in cur branch.
```

git branch

- The git branch command creates, lists and deletes branches
- It doesn’t allow switching between branches or putting a forked history back together again
- Git branches are a pointer to a snapshot of the changes you have made
- A new branch is created to encapsulate the changes when you want to fix bugs or add new features
- This helps you to clean up the future's history before merging it
- Git branches are an essential part of everyday workflow
- Git does not copy files from one directory to another, it stores the branch as a reference to a commit


master
commit1
branch1
commit4
commit2
branch2
commit3
commit5
merge1
commit6
commit7
merge2
commit8


HEAD
main
branch2
c: merge1
branch1
c: rectperi
c: multiply
c: rectarea
c: Sub fix
c: divide
c: subtr
c: add
c: file1


Server
files
db
data
4mb
meta data
100mb
git clone ①
git push ②
③ git clone
④ git push
⑤ git pull
git clone
git pull
dev1
dev2
dev3
files
db
data
meta data
github.com
OR
gitlab.com
OR
bitbucket.com
OR
aws Code Commit
OR
custom git

THANK YOU!

SUNBEAM INFOTECH